Teaching Objects Early and Design Patterns in Java Using Case Studies

Chris Nevison
Computer Science Dept.
Colgate University
Hamilton, NY 13346
chris@cs.colgate.edu

Barbara Wells
South Fork High School
10205 SW Pratt & Whitney Rd.
Stuart, FL 34997
bcw100@netzero.com

Abstract
In order to teach object-oriented design and programming in introductory computer science it is imperative to teach objects from the very beginning of the course. The use of interacting objects is motivated by examples with an inherent complexity. We suggest that a case study approach to teaching object-oriented programming can provide a context with simplicity within complexity, so that simple versions of the case study program or simple pieces of a more complex program can be used to teach concepts at an introductory level. A case study provides a setting where a progression of successively more sophisticated programs can be developed to introduce standard topics of the introductory course within an increasingly familiar context. At the same time, the design of these programs can illustrate some of the fundamental principles of object-oriented design as embodied in basic design patterns.

Categories and Subject Descriptors
D.1.5 Object-oriented programming

General Terms
Design, Languages

Keywords
Patterns, Objects

1. INTRODUCTION
Kristen Nygaard, one of the founders of object-oriented programming, promoted the use of complex examples to teach object-oriented design and programming right from the start[7, 8]. He used the example of a crowded restaurant with many interacting people as well as other objects. We believe that by using case studies we can introduce similarly complex examples early in an introductory course and use these examples to motivate an object-oriented approach to programming. To use a case study in this way we must chose a problem that can be simplified to present some of the basic concepts and then can be developed in a sequence of increasingly more complex programs to introduce a series of concepts appropriate to a first course.

ITiCSE'03, June 30–July 2, 2003, Thessaloniki, Greece.

One or more such case studies can be used along with smaller self-contained examples to teach the basic concepts in an introductory course. The case study will be somewhat intimidating to begin with, but as students work with a simple version followed by incrementally more complex versions, they will develop a familiarity with the context, so that they can focus on the new concepts presented rather than on the complexity of the problem. The secret is to find the simplicity that focuses on one concept within the complexity of the larger problem.

An important characteristic of a case study for the introductory course is that it use a graphical and interactive presentation. Today's students are visually oriented from their experience and are familiar with interactive, event-driven applications. An appropriate environment will be needed for such a presentation, so that students are not overwhelmed with the intricacies of event-driven graphical programming.

A well-chosen case study will facilitate the introduction of fundamental concepts in a coherent sequence, with the possibility of some flexibility to accommodate different approaches that teachers may use. It need not be used to demonstrate all the concepts taught, but it can be used for many. It will create a context with which students become increasingly familiar and that provides motivation for those concepts.

We will use one example of such a case study, an elevator control system, to demonstrate how it can be used and we will discuss some other case studies that can also be used in this context. We believe that a combination of interesting case studies can provide a large body of material for introductory computer science courses.

2. OTHER WORK
Case studies have long been an important component of teaching in general. Clancy and Lin [5,6], among others, have promoted the use of case studies for teaching introductory computer science. We are adapting this approach to the context of object-oriented programming and design patterns. In this context, we suggest that the use of case studies from the very beginning of the first course can be a successful strategy.

Several authors have also emphasized the importance of teaching "objects early" when teaching an object-oriented programming paradigm in the first computer science course. See, for example, [1], [3], [4], [7], [8] . We believe that "objects early" is quite natural for the o-o paradigm and that the only reason that there is any controversy is the tendency of all of us who teach to continue to use tried-and-true methods. Old habits die hard. However, we believe that Nygaard [7, 8] is right:

object-oriented design and programming is best understood for complex systems. Well-chosen case studies can provide the complexity to motivate object-oriented programming while also providing a context where concepts can be presented in a reasonably simple setting within the more complex environment.

Many of these same authors suggest that graphics is an important motivating tool for teaching introductory computer science and provide helpful programming environments for this approach [3], [10]. We will review three such environments in the next section.

Alphonce and Ventura advocate the use of design patterns in the first computer science course "to manage complexity" [1]. Their approach fits well with our case study driven approach to the introductory courses. Astrachan *et al.* also consider design patterns an essential part of the early computer science curriculum [2], as does Proulx [9].

3. INTERACTIVE GRAPHICS SYSTEMS

The combination of graphics and some degree of interactive, possibly event-driven, programming is a powerful motivation for students. However, the Java libraries for interactive graphical programs are quite complex and are too difficult for beginners. In order to incorporate graphics into our case studies, which we feel is imperative, we need to use a system designed to introduce the novice programmer to interactive graphical programming. BlueJ, Java Power Tools (JPT), and the objectdraw library are three such systems. BlueJ provides a GUI for displaying the class structure of programs and allowing one to create objects and invoke their methods without a driver -- a powerful learning tool. It also has a simple graphics library with some of the functionality needed to create our type of program, so it would be a reasonable tool to use (see http://www.BlueJ.org). JPT provides a system for making the creation of GUIs in general easier [10]. It is a powerful tool and could also be used to implement the examples we describe.

The objectdraw library is purposely limited so as to provide an introduction to graphics and objects suitable for the first course, with a transition to standard Java (or perhaps a more powerful tool like JPT) later in the course or in the second course (see http://applecore.cs.williams.edu/~cs134/eof/). We find that it suits our needs quite well, so we have developed our elevator case study using objectdraw. However, it could also be adapted to these other environments. The key is not the specific environment that is used, but that the environment simplify the development of interactive graphical objects for the student, so that the student can focus on the other concepts that are introduced. An effective environment will need some explanation up-front, but will soon recede from the foreground for the student, allowing concentration on other issues.

4. SEQUENCING TOPICS

A good case study will allow us to develop an oversimplified version of a program that captures some elements of the more complex problem while being simple enough to use early in an introductory course. It will also facilitate the development of incrementally more complex versions that demonstrate different new concepts, preferably one at a time.

The elevator case study works well. We can initially introduce

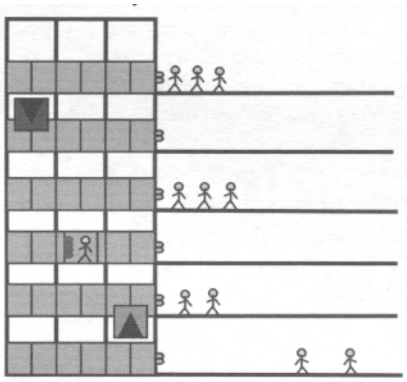

figure 1

the general problem with a diagram such as figure 1. This provides a context for discussing the motivation of object-oriented programming by identifying and discussing the various objects that are in the diagram -- people, elevators, buttons to call elevators to floors, buttons inside elevators, elevator doors, floors -- and their characteristics. (The elevator problem and the sequence of programs described here, and corresponding powerpoint slides are available in their entirety online at http://cs.colgate.edu/HSJava.html)

4.1 A first program

We can then simplify the problem of developing a control to make a graphical object representing an elevator move. By asking the "elevator" to move in one direction only, we can make the problem as simple as possible. In the context of the *objectdraw* library, the resulting code has a `begin` method (the *objectdraw* equivalent of `init` for applets) which simply creates an elevator as an instance of `FilledRect` (a black filled rectangle that is drawn by the *objectdraw* system as soon as it is constructed). The other method is `onMouseClick`, which overrides the same method as supplied by the `WindowController` class from *objectdraw*. `onMouseClick` provides event-driven functionality in a simplified form. The body of the `onMouseClick` method consists of a single statement, a call to the `FilledRect` method move that causes the elevator to move up in the applet window. We submit that this program, shown below, is hardly more complex than the typical boring "Hello World" program, but much more motivating for students as a first program. The complete code, with no comments, is shown below.

```
import objectdraw.WindowController;
import objectdraw.FilledRect;
import objectdraw.Location;

public class ElevatorSystem1
       extends WindowController{
  private FilledRect elevator;

  public void begin(){
    elevator = new FilledRect(100.0, 200.0,
                              20.0, 30.0,
                              canvas);
  }
  public void onMouseClick(Location point){
    elevator.move(0.0, -60.0);
  }
}
```

This example provides a context for discussing the main parts of a program and the simple operation of an event-driven program. It can lead to a simple exercise where the `elevator.move` call can be replaced by the use of `moveTo` that provides absolute movement and makes use of the formal parameter `point`.

4.2 Constructors, methods and parameters

After we review the general structure of a program, we can use this same simple example as the basis of our first discussion of instance variables that are objects and the need to construct these objects. This example demonstrates how parameters can be passed to a constructor to determine the characteristics of the object that is constructed. At this point we only work with classes from the *objectdraw* or standard Java libraries, so that we do not discuss how constructors are created.

We can use this same example to begin a discussion of how objects interact by calling methods (sending messages in o-o speak), and how methods often need to get some information when they are called -- parameters. We emphasize that any program exists within the context of a more complex system -- the runtime environment. It has been true for decades that programmers do not write programs that run by themselves -- they always run in the context of a programming environment and operating system. It is helpful to make that explicit from the beginning by discussing how there is a runtime environment that mediates between the physical events in the system -- keystrokes or mouse movements and click -- and the program that we write. In our event-driven example the `onMouseClick` method is invoked by this system, not by any component of our program, and the formal parameter is supplied a value by this invocation. We can separate the discussion of formal parameters that we first see being supplied by this outside force from the discussion of actual parameters (or arguments) that we supply when we call a method, as with the `elevator.move` call.

4.3 Introducing conditionals

Our primitive first example does not provide much control of the "elevator." It moves up on each mouse click until it disappears off the top of the window. We need to introduce a model of control into our system. We can do this by providing buttons that control the elevator. Initially we do this by simply drawing buttons and determining which is "pushed" with a mouse click by using a conditional. The *objectdraw* `FilledOval` provides the button objects and the `FilledOval` `boolean` method `contains` enables us to detect whether the mouse has been clicked within a specific button. Then a three-way conditional allows us to determine the appropriate action for the elevator. The picture drawn by this program is shown in figure 2, with the elevator at its starting position on the first floor.

figure 2

Although this kind of control is better done using a listener model, we submit that using a simple conditional is a reasonable for this level program and is a good motivation for introducing conditionals. After all, in the listener model the conditionals are there, they are just hidden in the library controls providing the interaction. We introduce this model later when the conditionals get so complex that the use of this more powerful tool is motivated and, indeed, is imperative to manage the complexity.

4.4 Specifying our own classes

We now introduce the idea of defining our own classes. Up to this point we have used classes supplied by the *objectdraw* library, `FilledRect` and `FilledOval`, that had a great deal of the functionality that we needed for our simpler models of elevator control. Now we would like an elevator that has an interior and doors that can open and close. Two other characteristics of an elevator is that it know which floor it is on and that it has a method for moving to floors that simply specifies which floor to move to, rather than how many pixels in the screen display it should move. In this way we are demonstrating how a class definition encapsulates details and provides a visible interface that fits the model being developed.

By drawing the elevator from several graphical components, each of which has its own move method (supplied by *objectdraw*), we have a good example of the *composite design pattern*. Although we do not believe that design patterns should be highly emphasized at this point, it is helpful to point them out and name them when they occur. This will help students get a reference to common patterns and will help them build an understanding of these patterns as they see them repeated in different contexts. In the code we write a private method for moving an elevator that simply calls the move method for each part of the elevator. We then write the `moveToLevel` method that takes the elevator to a specific floor using this composite move method and an appropriate constant for the distance between floors in pixels.

A simple exercise follows from this example. The doors of the elevator are represented by both `FilledRect` and `FramedRect` objects that open by a call to the `hide` methods for each component and close by a call to the `open` methods. It is quite natural to encapsulate this behavior in a door class, providing another example of the composite pattern.

4.5 Multiple elevators

The introduction of multiple elevators provides a place for reinforcing the ideas of developing our own classes for encapsulating behavior. A simple example shows that the previous model left out important components of elevator control and it does not make sense to have multiple elevators with only floor controls that call the closest elevator. For the three floors in the example, this results in only one elevator of the two ever moving. Other examples also have inappropriate behavior.

We need to add the other source of control for elevators to the model: the buttons inside the elevator that a rider would push to get taken to a certain floor. This leads to a model with both floor call buttons (not yet divided into up and down call buttons)

and panels of buttons representing the controls inside the elevators. A class representing a control panel is natural.

The resulting model has quite complex conditionals for determining which button has been "pushed" by a mouse click. There is an if ... else statement to detect whether one of the floor buttons has been called or whether one of the control panels was used. Within the control panel class there is another conditional to determine which button in the control panel was pushed, if this control panel was detected by the higher level conditional. This degree of complexity calls for a tool for simplification.

4.6 Listeners

We have added a feature to the *objectdraw* library, an `ODButton`, that is a simple form of a button class that has the capability of adding listeners that will be notified whenever certain mouse actions occur on an instance of this class -- mouse click, press, and release. An interface `ODButtonListener` and a class `ODButtonAdapter` provide the specification for a class that can be added as a listener to an `ODButton`. (We have provided a package called *objectdrawplus* that adds these components to the *objectdraw* library.) This enables us to introduce the *observer pattern*. Because the conditional detecting whether a particular button is touched by a mouse action is hidden in the library class `ODButton`, the code in our program is greatly simplified. In fact, we no longer need an `onMouseClick` method at all in the `ElevatorSystem` class nor a corresponding method in the control panel class.

Instead we need to define listeners that take the appropriate action when the button that they are attached to is "pushed." This means we need three control classes:

- one for the call buttons on each floor, parameterized by the particular floor;
- one for the buttons on the control panel that send that control panel's elevator to a particular floor, parameterized by elevator and floor;
- one for the open and close buttons, parameterized by whether the doors are to open or close and by elevator.

Since these button controls are each added to only one button, they can be declared anonymously in the program.

The use of the observer pattern and the corresponding tools are highly motivated in this example by the level of complexity and the simplification that they provide. It is not clear that the same case can be made for introducing them when only three buttons are involved. However, a simple exercise is to return to the three button model and redo it using the new model (or conversely, to demonstrate the tools there and do the more complex model as the exercise).

The need for conditionals has not disappeared, however. We still need to determine the closest elevator when a floor button is called, invoking the floor button control as a listener. This provides an opportunity for introducing another design pattern, the *expert pattern*. Which class should have the responsibility of determining which elevator is closest to the floor to which it has been called? In the model without the listeners, the conditional that calls the elevator is in the main `ElevatorSystem` class, so it is obvious to put a private helper method for determining the closest in that class. However in the new model with listeners, there are more simpler classes: the floor buttons which are instances of `ODButton`; the floor button control classes, the listeners that actually call the elevator; and the `ElevatorSystem` class is still there. The expert pattern suggests that the class with the requisite information have the responsibility. In this model, the `ElevatorSystem` class is the only one that knows about all the elevators and is therefore the "expert." Other ways of assigning this responsibility, for example to each of the floor button control classes, would require more complex patterns of communication. This is an example of how a design pattern can lead us to good solution.

4.7 Arrays

An obvious step as soon as we see that we might want more than three floors or more than two elevators is to introduce some means of keeping a list of floors and a list of elevators (and therefore elevator control panels). This is a natural motivation for introducing arrays.

Arrays are the appropriate data structure to use here since we know in advance the number of floors and elevators -- they are part of the structure that is determined when the system is created and will not change (unless the building collapses). Consequently, it is quite natural to introduce arrays of elevators, arrays of elevator control panels, arrays of floor call buttons (`ODButton` objects), and arrays of buttons within each control panel (also `ODButton` objects),.

4.8 Inheritance

Several elevators serving many floors might suggest that there could be different types of elevators such as express elevators. This is a natural place to introduce inheritance using our own classes. (We must use inheritance in a limited form right from the start if *objectdraw* is our tool -- the main program always extends the `WindowController` class that itself extends `Applet`. `WindowController` hides the setup of graphics and provides a simplified model of events.)

An express elevator has a designated level and only serves the first floor and floors at or above the designated level. This is really a matter of control, so we define an express elevator control panel class that extends the elevator control panel class.

There are different exercises that can be used to provide additional examples of inheritance. Different types of elevator control can be added. An interesting inheritance exercise is to define an elevator button by extending the `ODButton` class. An elevator button is an `ODButton` that is always round and yellow. It also always has a number or a label.

4.9 Extensions

The elevator model can be extended to be a more complete simulation of an elevator system. This moves away from the problem of modeling the control system in an interactive program to building a simulation model. The graphics would be a display of the underlying simulation model. This provides the possibility of a more complex program that can reinforce many of the concepts already introduced and also add some additional concept, such as various data structures. Additional design

patterns, such as the *model-view-controller pattern* are likely to come up in this more complex system.

5. DESIGN PATTERNS

We believe that design patterns can help students understand the organization of a large program. We do not expect beginning students to memorize design patterns or explicitly evaluate designs, much less make decisions about design based on design patterns. However, they can be a useful organizing principle as they read programs that involve interacting classes and ask the inevitable question, "Why do it this way?" As students progress into the second course in computer science, they will begin to develop larger programs of their own, and this exposure to design patterns can help provide them with some organizing principles.

In our elevator control system case study, we point out design patterns as they occur naturally, and indicate how they summarize useful principles behind the design of the program. The three that we have mentioned above, *composite*, *observer*, and *expert*, could certainly be supplemented.

6. OTHER CASE STUDIES

We would not expect to develop a course using only one case study. An example that provides context that runs throughout a course is a powerful motivating tool. However, some students may find a different example more compelling. We recommend using two or three case studies to which one can return repeatedly in order to teach new concepts or to reinforce concepts taught using other examples.

We are in the process of developing a case study based on finding the path through a maze. This material has already been developed in C++ and most of it will readily carry over to Java. In addition to providing a vehicle where some basic programming can be taught, it also provides a platform where more work with algorithms can be incorporated. Thus it is a case study that can carry through from a first course into many of the topics that would be covered in the typical second course.

The maze case study progression will include basic programming ideas first illustrated with a random walk, without a maze, followed by an interactive program controlling the walk that demonstrates some basic control structures. Later the maze itself will provide a platform for discussing alternate data structures and recursion. Finally, systematic solution of the maze using stacks for depth-first search, queues for breadth first search, and priority queues for heuristic best-first search provides some rich examples of algorithms that are quite appropriate to a second Computer Science course.

Another rich case study has been developed by the College Board for the Advanced Placement Computer Science program. This case study is a "Marine Biology Simulation" involving the study of fish movement and also fish breeding and dying. This case study builds from material that can be used early in the first course, through material that provides excellent examples for a data structures course. In addition, some of the classes developed in this case study can be used in other contexts. (See http://apcentral.collegeboard.com.)

7. SUMMARY

We have presented an example demonstrating how one can use case studies to teach introductory computer science from the very beginning. We have had great success with this approach at workshops for teachers and several of those teachers are using our ideas in their classes now.

The advantages we see to using case studies include the following:

- Complexity in a controlled situation
- Introduction of objects early
- Incremental introduction of topics
- Provide a familiar context for students throughout a course
- Provide a realistic context for the application of concepts
- Provide a context for introducing design patterns

8. REFERENCES

[1] Alphonce, Carl, and Phil Ventura, 2002. "Object-Orientation in CS1-CS2 by Design," *Proceedings of the 7th Annual Conference on Innovation and Technology in computer Science Education* (ITiCSE 2002), 70-74.

[2] Astrachan, Owen, Geoffrey Berry, Landon Cox, Garrett Mitchener, 1998. "Design Patterns: An Essential Component of CS Curricula," *ACM SIGCSE Bulletin* v 30 n 1, 153-160.

[3] Bruce, Kim, Andrea Danyluk, Thomas Murtagh, 2001. "A Library to Support a Graphics-Based Objects-First Approach to CS 1," *ACM SIGCSE Bulletin* v33 n1, 6-10.

[4] Christensen, Henrik, and Michael Caspersen, 2002. "Frameworks in CS1 - a Different Way of Introducing Event-driven Programming," *Proceedings of the 7th Annual Conference on Innovation and Technology in computer Science Education* (ITiCSE 2002), 75-79.

[5] Clancy, Michael, and Marcia Lin, 1992. "Case Studies in the Classroom," *ACM SIGCSE Bulletin* v 24 n 1, 220-224.

[6] ____, 1992. *Designing Pascal Solutions: A Case Study Approach*, W.H. Freeman: New York.

[7] Nygaard, Kristen, 2001. Invited Talk OOPSLA'01, Educators' Symposium.

[8] Nygaard, Kristen, 2001. "COOL (Comprehensive Object-Oriented Learning," (keynote address) *Proceedings of the 7th Annual Conference on Innovation and Technology in computer Science Education* (ITiCSE 2002), 218.

[9] Proulx, Viera, 2000. "Programing Patterns and Design Patterns in the Introductory Computer Science Course," *ACM SIGCSE Bulletin* v 32 n 1, 80-84.

[10] Proulx, Viera, Jeff Raab, Richard Rasala, 2002. "Objects from the Beginning," *Proceedings of the 7th Annual Conference on Innovation and Technology in computer Science Education* (ITiCSE 2002), 70-74.